

DESIGN AND ANALYSIS OF ALGORITHMS

Complete Study Guide — CS3004-1

NITTE · B.Tech. Computer Science & Engineering

Based on: Levitin — Introduction to the Design & Analysis of Algorithms (3rd Ed.)

Unit I · Unit II · Unit III

UNIT I

15 Hrs

Intro · Analysis · Brute Force · Divide &
Conquer

UNIT II

15 Hrs

Decrease·Transform · Space-Time ·
Dynamic Prog.

UNIT III

10 Hrs

Greedy · Backtracking · Branch &
Bound · P/NP

TABLE OF CONTENTS

UNIT I — Introduction, Analysis & Divide-and-Conquer (15 Hrs)

- 1. What is an Algorithm? — Properties, Examples, Pseudocode
- 2. Fundamentals of Algorithmic Problem Solving
- 3. Analysis Framework — Time & Space Complexity
- 4. Asymptotic Notations — O , Ω , Θ , Efficiency Classes
- 5. Analysis of Non-Recursive Algorithms
- 6. Analysis of Recursive Algorithms & Recurrences
- 7. Brute Force — Selection Sort, Bubble Sort, String Matching
- 8. Divide-and-Conquer — Merge Sort, Quick Sort, Binary Search
- 9. Strassen's Matrix Multiplication

UNIT II — Decrease, Transform, Trade-offs & Dynamic Programming (15 Hrs)

- 10. Decrease-and-Conquer — Insertion Sort, Binary Search
- 11. Graph Algorithms — DFS, BFS, Topological Sort
- 12. Transform-and-Conquer — AVL Trees, 2-3 Trees, Heaps & Heapsort
- 13. Space-Time Tradeoffs — Hashing, Horspool's String Matching
- 14. Dynamic Programming — Principle, Fibonacci, Knapsack
- 15. Floyd-Warshall All-Pairs Shortest Paths

UNIT III — Greedy, Backtracking & Limitations (10 Hrs)

- 16. Greedy Technique — Prim's, Kruskal's MST
- 17. Dijkstra's Shortest Path & Huffman Trees
- 18. Backtracking — N-Queens, Subset-Sum, Graph Coloring
- 19. Branch and Bound — Assignment, Knapsack, TSP
- 20. P, NP and NP-Complete Problems

UNIT I

Introduction · Analysis Framework · Brute Force · Divide-and-Conquer

1. What is an Algorithm?

An **algorithm** is a sequence of unambiguous instructions for solving a problem — i.e., for obtaining a required output for any legitimate input in a finite amount of time. The word comes from the name of 9th-century mathematician *al-Khwarizmi*.

Key Properties of an Algorithm

1. Input — zero or more quantities externally supplied
2. Output — at least one quantity is produced
3. Definiteness — each step is clear and unambiguous
4. Finiteness — terminates after a finite number of steps
5. Effectiveness — each step is basic enough to be carried out exactly

Classic Example: Euclid's GCD Algorithm

Euclid's algorithm computes $\text{gcd}(m, n)$ by repeatedly applying: **$\text{gcd}(m, n) = \text{gcd}(n, m \bmod n)$** until $m \bmod n = 0$.

```
ALGORITHM Euclid(m, n)
// Input: Two non-negative integers m and n
// Output: gcd(m, n)
while n ≠ 0 do
  r ← m mod n
  m ← n
  n ← r
return m
```

★ Example: $\text{gcd}(60, 24) \rightarrow \text{gcd}(24, 12) \rightarrow \text{gcd}(12, 0) = 12$

Algorithm Design & Analysis Process

Step 1	Understand the problem — read carefully, check edge cases
Step 2	Decide on computational means — exact vs. approximate solving
Step 3	Choose an algorithm design technique
Step 4	Design the algorithm and its data structures
Step 5	Prove correctness of the algorithm
Step 6	Analyze the algorithm (time & space)
Step 7	Code the algorithm

Important Problem Types

Sorting	Rearrange items in non-decreasing order. Key: stability, in-place.
Searching	Find a given value in a set. Key: sorted vs. unsorted input.
String Processing	Pattern matching, text compression, bioinformatics.
Graph Problems	Graph traversal, shortest paths, spanning trees, topological sort.
Combinatorial	Finding permutations/subsets satisfying constraints — often NP-hard.
Geometric	Points, lines, polygons — closest pair, convex hull.
Numerical	Equations, integrals, matrices — round-off errors matter.

2. Fundamentals of Analysis — Efficiency Framework

Algorithm efficiency is measured primarily by **time complexity** (how fast) and **space complexity** (how much memory). We measure time by counting the number of times the **basic operation** (innermost loop) is executed as a function of input size n .

Worst, Best, Average Case

Case	Notation	Definition	Usage
Worst	$T_{\text{worst}}(n)$	Maximum over all inputs of size n	Guarantees upper bound
Best	$T_{\text{best}}(n)$	Minimum over all inputs of size n	Optimistic bound
Average	$T_{\text{avg}}(n)$	Expected over all random inputs	Probabilistic analysis

3. Asymptotic Notations

$O(g(n))$ — Big-O	UPPER bound. $t(n) \leq c \cdot g(n)$ for all $n \geq n_0$. Worst-case ceiling.
$\Omega(g(n))$ — Big-Omega	LOWER bound. $t(n) \geq c \cdot g(n)$ for all $n \geq n_0$. Best-case floor.
$\Theta(g(n))$ — Big-Theta	TIGHT bound. $c_1 \cdot g(n) \leq t(n) \leq c_2 \cdot g(n)$. Both upper and lower.
$o(g(n))$ — Little-o	Strictly less: $\lim t(n)/g(n) = 0$ as $n \rightarrow \infty$

Efficiency Classes (Slowest to Fastest Growing)

Class	Name	Example Algorithms
$O(1)$	Constant	Array access, stack push/pop

$O(\log n)$	Logarithmic	Binary search
$O(n)$	Linear	Sequential search, traversal
$O(n \log n)$	Linearithmic	Merge sort, Heap sort
$O(n^2)$	Quadratic	Bubble sort, Selection sort, Insertion sort
$O(n^3)$	Cubic	Matrix multiplication (naive)
$O(2^n)$	Exponential	Exhaustive search, Tower of Hanoi
$O(n!)$	Factorial	Permutation generation

★ Limit Rule: if $\lim[t(n)/g(n)] = 0 \rightarrow t \in o(g)$; $= c > 0 \rightarrow t \in \Theta(g)$; $= \infty \rightarrow t \in \omega(g)$

4. Analysis of Non-Recursive Algorithms

General plan for non-recursive algorithm analysis:

- 1. Decide on input size parameter n
- 2. Identify the basic operation (usually in innermost loop)
- 3. Check if count depends only on n , or also on input specifics
- 4. Set up a sum for the count of basic operations
- 5. Use summation formulas to find a closed-form solution

Example: Maximum Element

```

ALGORITHM MaxElement(A[0..n-1])
maxval ← A[0]
for i ← 1 to n-1 do
  if A[i] > maxval ← Basic Operation (comparison)
  maxval ← A[i]
return maxval

```

$C(n) = \sum_{i=1}^{n-1} 1 = n - 1 \rightarrow \Theta(n)$

5. Analysis of Recursive Algorithms

For recursive algorithms, set up a **recurrence relation** and solve it.

General Plan:

- 1. Identify input size parameter n
- 2. Identify basic operation
- 3. Set up recurrence relation $T(n)$ with initial condition

■ 4. Solve recurrence by backward substitution or Master Theorem

Master Theorem

Master Theorem — $T(n) = aT(n/b) + f(n)$

Case 1: $f(n) = O(n^{\log_b a - \epsilon}) \rightarrow T(n) = \Theta(n^{\log_b a})$

Case 2: $f(n) = \Theta(n^{\log_b a}) \rightarrow T(n) = \Theta(n^{\log_b a} \cdot \log n)$

Case 3: $f(n) = \Omega(n^{\log_b a + \epsilon}) \rightarrow T(n) = \Theta(f(n))$

Common Recurrences

Recurrence	Solution	Example
$T(n) = T(n-1) + c$	$\Theta(n)$	Linear search
$T(n) = T(n/2) + c$	$\Theta(\log n)$	Binary search
$T(n) = 2T(n/2) + n$	$\Theta(n \log n)$	Merge sort
$T(n) = 2T(n-1) + 1$	$\Theta(2^n)$	Tower of Hanoi
$T(n) = T(n-1) + n$	$\Theta(n^2)$	Selection sort

6. Brute Force & Exhaustive Search

Brute force applies the problem definition directly without optimisation. Simple to implement but often inefficient. Exhaustive search evaluates all candidates.

Selection Sort

Find the minimum in the unsorted part, swap it to the front. Repeat.

```
ALGORITHM SelectionSort(A[0..n-1])
for i ← 0 to n-2 do
  min ← i
  for j ← i+1 to n-1 do
    if A[j] < A[min] min ← j
  swap A[i] and A[min]
```

$C(n) = (n-1) + (n-2) + \dots + 1 = n(n-1)/2 \rightarrow \Theta(n^2)$

★ In-place, not stable. Comparisons: $\Theta(n^2)$. Swaps: $O(n)$.

Bubble Sort

Repeatedly compare adjacent elements, swapping if out of order. After each pass, the largest unsorted element "bubbles up".

```
ALGORITHM BubbleSort(A[0..n-1])
for i ← 0 to n-2 do
  for j ← 0 to n-2-i do
    if A[j+1] < A[j] swap A[j] and A[j+1]
```

Worst/Average: $\Theta(n^2)$ Best (already sorted): $\Theta(n)$ with flag

Sequential Search

```
ALGORITHM SequentialSearch(A[0..n-1], K)
for i ← 0 to n-1 do
  if A[i] = K return i
return -1
```

Best: $O(1)$ Worst/Average: $O(n)$

Brute-Force String Matching

Slide pattern $P[0..m-1]$ over text $T[0..n-1]$, checking character by character.

```
ALGORITHM BruteForceStringMatch(T[0..n-1], P[0..m-1])
for i ← 0 to n-m do
  j ← 0
  while j < m and P[j] = T[i+j]
    j ← j + 1
  if j = m return i
return -1
```

Worst case: $\Theta((n-m+1) \cdot m) \approx \Theta(n \cdot m)$

7. Divide-and-Conquer

Divide-and-Conquer Paradigm

1. DIVIDE — Split problem into smaller sub-problems of same type
2. CONQUER — Solve sub-problems recursively
3. COMBINE — Merge sub-problem solutions into overall solution

$T(n) = aT(n/b) + f(n)$ — solved by Master Theorem

Merge Sort

Divide array in half, recursively sort each half, merge the two sorted halves.

```
ALGORITHM MergeSort(A[0..n-1])
if n > 1
  copy A[0..n/2-1] to B; copy A[n/2..n-1] to C
  MergeSort(B); MergeSort(C)
  Merge(B, C, A)
```

$T(n) = 2T(n/2) + n \rightarrow \Theta(n \log n)$ [all cases]
Space: $O(n)$ extra | Stable: Yes

★ Merge step: compare heads of two sorted sub-arrays, always pick smaller.

Quick Sort

Select a pivot, partition array around pivot (smaller left, larger right), recursively sort partitions.

```
ALGORITHM QuickSort(A[l..r])
if l < r
  s ← Partition(A[l..r]) // s is pivot position
  QuickSort(A[l..s-1])
  QuickSort(A[s+1..r])
```

Best/Average: $\Theta(n \log n)$ Worst (sorted input): $\Theta(n^2)$
In-place: Yes | Stable: No

Binary Search

Search in a sorted array by repeatedly halving the search space.

```
ALGORITHM BinarySearch(A[0..n-1], K)
l ← 0; r ← n-1
while l ≤ r do
m ← (l+r)/2
if K = A[m] return m
else if K < A[m] r ← m-1
else l ← m+1
return -1

T(n) = T(n/2) + 1 →  $\Theta(\log n)$ 
```

Strassen's Matrix Multiplication

Multiply two $n \times n$ matrices using 7 multiplications (vs 8 in standard divide-and-conquer). Each $n \times n$ multiplication reduced to seven $n/2 \times n/2$ multiplications.

```
Standard D&C: T(n) = 8T(n/2) + n^2 →  $\Theta(n^3)$ 
Strassen: T(n) = 7T(n/2) + n^2 →  $\Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$ 

M1 = (A00+A11)(B00+B11)
M2 = (A10+A11)B00
M3 = A00(B01-B11)
M4 = A11(B10-B00)
M5 = (A00+A01)B11
M6 = (A10-A00)(B00+B01)
M7 = (A01-A11)(B10+B11)
```

Comparison: Sorting Algorithms (Unit I)

Algorithm	Best	Average	Worst	Space	Stable
Selection Sort	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	No
Bubble Sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Yes
Merge Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(n)$	Yes
Quick Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$	$O(\log n)$	No

UNIT II

Decrease-and-Conquer · Transform-and-Conquer · Space-Time Tradeoffs · Dynamic Programming

8. Decrease-and-Conquer

Decrease-and-Conquer Idea

Reduce problem instance to a SMALLER instance of the SAME problem.

Types: Decrease-by-constant ($n \rightarrow n-1$), Decrease-by-factor ($n \rightarrow n/b$), Variable-size-decrease.

Key difference from D&C: only ONE smaller sub-problem (not multiple).

Insertion Sort

Insert the i -th element into its correct position among the already-sorted first $i-1$ elements.

```
ALGORITHM InsertionSort(A[0..n-1])
for i ← 1 to n-1 do
  v ← A[i]
  j ← i - 1
  while j ≥ 0 and A[j] > v do
    A[j+1] ← A[j]
    j ← j - 1
  A[j+1] ← v
```

Best: $\Theta(n)$ Worst/Average: $\Theta(n^2)$ | In-place, Stable

9. Graph Algorithms — DFS, BFS & Topological Sort

Depth-First Search (DFS)

Explore as deep as possible before backtracking. Uses a stack (or recursion). Builds DFS tree; marks discovery order (push time) and finish order (pop time).

```
ALGORITHM DFS(Graph G)
mark all vertices as unvisited
for each vertex v do
  if v is unvisited
    dfs(v)
```

```
ALGORITHM dfs(v)
mark v as visited
for each neighbour w of v do
  if w is unvisited dfs(w)
```

Time: $\Theta(V+E)$ for adj. list | $\Theta(V^2)$ for adj. matrix

★ DFS applications: cycle detection, topological sort, connected components, articulation points.

Breadth-First Search (BFS)

Explore level by level using a queue. Finds shortest paths in unweighted graphs.

```
ALGORITHM BFS(Graph G, start s)
mark s as visited; enqueue s
while queue not empty do
v ← dequeue
for each neighbour w of v do
if w not visited
mark w visited; enqueue w

Time:  $\Theta(V+E)$  adj. list |  $\Theta(V^2)$  adj. matrix
```

Topological Sort

Linear ordering of vertices of a DAG such that for every directed edge (u,v), u comes before v. Used for scheduling, dependency resolution.

Method 1 — DFS-based:

```
Run DFS; output vertices in reverse finish time order.
Time:  $\Theta(V+E)$ 
```

Method 2 — Kahn's Algorithm (Source Removal):

```
1. Compute in-degree of all vertices
2. Enqueue all vertices with in-degree 0
3. While queue not empty:
  dequeue vertex v, add to result
  for each neighbour w: decrement in-degree[w]
  if in-degree[w] = 0: enqueue w
Time:  $\Theta(V+E)$ 
```

★ If topological sort cannot complete (cycle detected), the graph is NOT a DAG.

10. Transform-and-Conquer — AVL Trees, 2-3 Trees, Heaps

AVL Trees (Balanced BST)

A BST where for every node, the heights of left and right subtrees differ by at most 1 (balance factor $\in \{-1, 0, 1\}$). Guarantees $O(\log n)$ for search, insert, delete.

LL Rotation (Right)	Right-heavy left child. Single right rotation at root.
RR Rotation (Left)	Left-heavy right child. Single left rotation at root.
LR Rotation	Right-heavy left child. Left rotate child, then right rotate root.
RL Rotation	Left-heavy right child. Right rotate child, then left rotate root.

★ Height of AVL tree with n nodes: $h \leq 1.44 \log_2(n+2) - 1.33$

2-3 Trees

A balanced search tree where every internal node has 2 or 3 children, and all leaves are at the same level. Insertion may cause node splitting.

2-node	Has 1 key and 2 children (standard BST node)
3-node	Has 2 keys ($k_1 < k_2$) and 3 children; left < k_1 < middle < k_2 < right

Insertion	Find leaf, insert key; if 3-node overflows → split and push median up
Search	$O(\log n)$ — tree is always perfectly balanced

Heaps & Heapsort

Heap Definition

A max-heap is a complete binary tree where each node $\geq$ its children.

Stored in array $A[1..n]$: parent of $A[i]$ is $A[\lfloor i/2 \rfloor]$; children are $A[2i]$ and $A[2i+1]$.

Max element is always at root $A[1]$.

```

ALGORITHM Heapsort( $A[1..n]$ )
// Phase 1: Build max-heap from bottom up —  $O(n)$ 
for  $i \leftarrow \lfloor n/2 \rfloor$  downto 1 do HeapifyDown( $A, i, n$ )

// Phase 2: Extract max  $n-1$  times —  $O(n \log n)$ 
for  $i \leftarrow n$  downto 2 do
  swap  $A[1]$  and  $A[i]$ 
  HeapifyDown( $A, 1, i-1$ )

 $\Theta(n \log n)$  all cases | In-place | Not stable

```

11. Space-Time Tradeoffs

Use extra space (pre-computation, lookup tables) to speed up time. Two main techniques: **input enhancement** (preprocess input) and **prestructuring** (use extra data structure).

Hashing

Store keys in a hash table using a hash function $h(k) = k \bmod m$. Supports average-case $O(1)$ search, insert, delete.

Open Hashing (Chaining)	Each slot is a linked list. Simple, handles high load.
Closed Hashing (Open Addressing)	All keys in table. Linear/quadratic probing or double hashing.
Load factor $\alpha = n/m$	When $\alpha > 0.7$, performance degrades; rehash to larger table.

Horspool's String Matching

Preprocessing-based string matching. Pre-compute shift table from pattern to skip non-matching positions efficiently.

```

Preprocessing — Build Shift Table  $t(c)$ :
for each character  $c$  in alphabet:
   $t(c) = m$  (default = pattern length)
for  $j = 0$  to  $m-2$ :
   $t(P[j]) = m-1-j$ 

Search:
 $i \leftarrow m-1$ 
while  $i \leq n-1$ :
   $k \leftarrow 0$ 

```

```

while k ≤ m-1 and P[m-1-k] = T[i-k]: k++
if k = m: match at i-m+1
i ← i + t(T[i])

```

Average: $\Theta(n/m)$ | Worst: $\Theta(nm)$

12. Dynamic Programming

Dynamic Programming Principle

Break problem into overlapping sub-problems.

Store sub-problem solutions in a TABLE (memoization / bottom-up tabulation).

Two conditions: (1) Optimal Substructure (2) Overlapping Sub-problems.

Bottom-up: fill table in order of increasing sub-problem size.

0/1 Knapsack Problem

Given n items each with weight w_i and value v_i , and capacity W , maximize total value without exceeding capacity.

Let $V[i][w]$ = max value using items $1..i$ with capacity w

Recurrence:

$V[0][w] = 0$ for all w

$V[i][w] = V[i-1][w]$ if $w_i > w$

$\max(V[i-1][w], v_i + V[i-1][w-w_i])$ otherwise

Time: $\Theta(nW)$ Space: $\Theta(nW)$ (pseudo-polynomial)

Floyd-Warshall All-Pairs Shortest Paths

Find shortest paths between ALL pairs of vertices in a weighted directed graph. Handles negative weights (but not negative cycles).

ALGORITHM Floyd($W[1..n, 1..n]$)

$D \leftarrow W$ // $D[i][j]$ = direct edge weight (∞ if no edge)

for $k \leftarrow 1$ to n do

for $i \leftarrow 1$ to n do

for $j \leftarrow 1$ to n do

$D[i][j] \leftarrow \min(D[i][j], D[i][k] + D[k][j])$

return D

Time: $\Theta(n^3)$ Space: $\Theta(n^2)$

Interpretation: $D[i][j]$ = shortest path from i to j

★ Warshall's algorithm (same structure) finds transitive closure of a graph.

Warshall's Algorithm (Transitive Closure)

ALGORITHM Warshall($A[1..n, 1..n]$)

$R \leftarrow A$

for $k \leftarrow 1$ to n do

for $i \leftarrow 1$ to n do

for $j \leftarrow 1$ to n do

```
R[i][j] ← R[i][j] OR (R[i][k] AND R[k][j])  
return R
```

```
R[i][j] = 1 if there is a path from i to j, 0 otherwise
```

UNIT III

Greedy Technique · Backtracking · Branch and Bound · P, NP & NP-Complete

13. Greedy Technique

Greedy Strategy

At each step, choose the LOCALLY OPTIMAL choice hoping it leads to global optimum.

Does NOT reconsider previous choices.

Works when problem has: (1) Greedy-choice property (2) Optimal substructure.

Simpler and faster than DP, but not always correct.

Prim's Algorithm — Minimum Spanning Tree

Grow MST by adding the cheapest edge connecting the tree to a non-tree vertex.

```
ALGORITHM Prim(G, r) // r = starting vertex
Initialize all keys to  $\infty$ ; key[r] = 0; parent[r] = NIL
Q = all vertices
while Q  $\neq \emptyset$ :
    u = ExtractMin(Q)
    for each neighbour v of u:
        if v  $\in$  Q and w(u,v) < key[v]:
            parent[v] = u; key[v] = w(u,v)

With binary heap:  $O((V+E) \log V)$ 
With Fibonacci heap:  $O(E + V \log V)$ 
```

Kruskal's Algorithm — Minimum Spanning Tree

Sort edges by weight; add edge if it does not form a cycle (using Union-Find).

```
ALGORITHM Kruskal(G)
Sort all edges by weight
Initialize Union-Find (each vertex its own component)
MST = {}
for each edge (u,v) in sorted order:
    if Find(u)  $\neq$  Find(v): // no cycle
        MST = MST  $\cup$  {(u,v)}
    Union(u, v)
return MST

Time:  $O(E \log E)$  dominated by sorting
```

Dijkstra's Algorithm — Single-Source Shortest Paths

Find shortest paths from source s to all vertices. Requires NON-NEGATIVE weights.

```
ALGORITHM Dijkstra(G, s)
dist[s] = 0; dist[v] =  $\infty$  for all v  $\neq$  s
Q = all vertices
while Q  $\neq \emptyset$ :
```

```

u = vertex in Q with min dist[u]
remove u from Q
for each neighbour v of u:
if dist[u] + w(u,v) < dist[v]:
dist[v] = dist[u] + w(u,v)
prev[v] = u

```

With binary heap: $O((V+E) \log V)$
 With adj. matrix: $O(V^2)$

★ Bellman-Ford (not greedy) handles negative weights: $O(VE)$.

Huffman Trees & Codes

Build optimal prefix-free variable-length codes by greedily merging the two lowest-frequency symbols.

```

ALGORITHM HuffmanCode(C) // C = set of (symbol, freq) pairs
Insert all symbols into min-priority queue Q by freq
for i = 1 to |C|-1:
z = new node
z.left = x = ExtractMin(Q)
z.right = y = ExtractMin(Q)
z.freq = x.freq + y.freq
Insert(Q, z)
return ExtractMin(Q) // root of Huffman tree

```

Time: $O(n \log n)$ | Optimal prefix-free code

14. Backtracking

Backtracking Concept

Systematically explore all candidate solutions using recursion.

Prune (abandon) branches when a partial solution cannot lead to a valid solution.

Works on a State Space Tree — each node represents a partial candidate.

More efficient than exhaustive search due to pruning.

N-Queens Problem

Place N queens on an $N \times N$ chessboard so no two queens attack each other (no same row, column, or diagonal).

```

ALGORITHM NQueens(row)
if row > N: print solution; return
for col = 1 to N:
if isSafe(row, col):
place queen at (row, col)
NQueens(row + 1)
remove queen from (row, col)

```

isSafe checks: no queen in same column or diagonal

Subset-Sum Problem

Find all subsets of a given set $\{a_1, \dots, a_n\}$ that sum to exactly d .

```
ALGORITHM SubsetSum(i, currentSum, remaining)
if currentSum = d: print subset; return
if i > n or currentSum > d: return

// Include a[i]
SubsetSum(i+1, currentSum + a[i], remaining - a[i])

// Exclude a[i] – prune if remaining too small
if currentSum + remaining >= d:
    SubsetSum(i+1, currentSum, remaining - a[i])
```

15. Branch and Bound

Branch and Bound vs Backtracking

Branch and Bound: for OPTIMIZATION problems (min/max).

Uses a BOUND function to prune subtrees that cannot improve current best solution.

Explores nodes in best-first order (using priority queue) for efficiency.

Backtracking: for feasibility/enumeration; B&B: for optimization.

Assignment Problem (B&B;)

Assign n jobs to n workers (one each) minimizing total cost.

Lower bound for a partial assignment:
Cost so far + (min cost for each unassigned job from remaining workers)

Algorithm:

1. Start: root = no assignments, compute lower bound
2. Branch on next job assignment
3. Compute lower bound for each child node
4. Prune if lower bound $\geq$ current best solution
5. Update best when complete assignment found

Knapsack (B&B; — Maximization)

Sort items by value/weight ratio (descending)
Upper bound = value so far + fractional item to fill remaining capacity
Branch: include item i or exclude item i
Prune: if upper bound $\leq$ best value found so far

Travelling Salesman (B&B;)

Lower bound for partial tour:
Sum of edges so far + lower bound for completing the tour
Classic bound: for each vertex, add half of two cheapest edges
Branch on next city to visit; prune when $LB \geq$ best complete tour

16. P, NP, and NP-Complete Problems

P

Class of decision problems solvable in polynomial time $O(n^k)$. "Easy" problems.

NP	Non-deterministic Polynomial time. Problems VERIFIABLE in polynomial time.
NP-Hard	At least as hard as hardest problems in NP. May not be in NP.
NP-Complete	In NP AND NP-Hard. Both verifiable and as hard as any NP problem.
P = NP?	The greatest open problem in CS. Most believe $P \neq NP$ (unproven).

Key NP-Complete Problems

Problem	Description
SAT (Satisfiability)	Can boolean formula be made true? (Cook's theorem — first NPC)
3-SAT	SAT with clauses of exactly 3 literals each
Hamiltonian Circuit	Does a graph have a tour visiting every vertex exactly once?
Travelling Salesman	Find shortest Hamiltonian circuit in weighted graph
Vertex Cover	Find min set of vertices covering all edges
Clique	Does graph have a complete subgraph of size k ?
Knapsack (0/1)	Decision version: can we achieve value $\geq V$ within weight W ?
Graph Coloring	Can we color graph with k colors so no adjacent same color?

Reductions

Problem A reduces to B ($A \leq_p B$) means: if we can solve B in polynomial time, we can solve A in polynomial time. To prove C is NP-Complete: (1) show $C \in NP$, (2) show known NPC problem $\leq_p C$.

Quick Reference — Algorithm Complexity Summary

Algorithm	Technique	Best	Average	Worst	Space
Selection Sort	Brute Force	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$
Bubble Sort	Brute Force	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$
Insertion Sort	Decrease & C	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$
Merge Sort	Divide & C	$\Theta(n \lg n)$	$\Theta(n \lg n)$	$\Theta(n \lg n)$	$O(n)$
Quick Sort	Divide & C	$\Theta(n \lg n)$	$\Theta(n \lg n)$	$\Theta(n^2)$	$O(\lg n)$
Heap Sort	Transform & C	$\Theta(n \lg n)$	$\Theta(n \lg n)$	$\Theta(n \lg n)$	$O(1)$
Binary Search	Divide & C	$O(1)$	$O(\lg n)$	$O(\lg n)$	$O(1)$
Dijkstra's	Greedy	-	$O((V+E)\lg V)$	$O((V+E)\lg V)$	$O(V)$
Prim's	Greedy	-	$O((V+E)\lg V)$	$O((V+E)\lg V)$	$O(V)$
Kruskal's	Greedy	-	$O(E \lg E)$	$O(E \lg E)$	$O(V)$
Floyd-Warshall	Dynamic Prog	-	$\Theta(n^3)$	$\Theta(n^3)$	$\Theta(n^2)$
Knapsack DP	Dynamic Prog	-	$\Theta(nW)$	$\Theta(nW)$	$\Theta(nW)$
Horspool's	Space-Time	-	$\Theta(n/m)$	$\Theta(nm)$	$O(m+ \Sigma)$